

M-AI Master

技术说明文档 V4.3

主数据智能治理与智能共享 · 比赛与企业交付增强版

面向大型制造集团多系统异构主数据治理，构建大模型标准识别、真实图文填报、持久化向量检索、图谱辅助查重、可信审计、全流程审批、跨工厂分发与知识回流的一体化治理 Agent。

原始记录	黄金主数据	候选压缩率	回归测试
234	72	69.2%	24/24

目录

1. 文档说明与验证边界
2. 执行摘要
3. 比赛要求逐项对照
4. 业务痛点与建设目标
5. 总体架构
6. 主数据治理 Agent 八步闭环
7. AI 语义识别引擎
8. 标准识别、分类推荐与图文填报
9. AI 相似度查重与黄金主数据生成
10. 全流程自动化与可审计治理
11. 多工厂多主体协同治理
12. 自然语言驱动跨系统分发
13. 数据可信、向量知识与系统韧性
14. 量化结果与业务价值
15. 二十项比赛亮点
16. REST API 与模块映射
17. 部署、配置与验证
18. 企业交付现状与演进路线
19. 核心实现代码
20. 测试证据与代码定位

文档说明与验证边界

本文档是 M-AI Master 4.3 当前可运行系统的比赛与企业交付增强版技术说明，依据 Flask 后端、单页前端、SQLite 权威数据库、隔离式 PaddleOCR 运行时、50 条人工真值集和 24 项自动化测试更新。文档重点回答四个问题：AI 是否真实参与语义识别，主数据是否形成可回放闭环，多工厂价值是否可验证，系统是否具备跨电脑与单节点企业试点交付能力。

项目	当前事实
核心实现	Python + Flask + SQLite；单页 HTML；IndexedDB 仅作为缓存
AI 引擎	通义 text-embedding-v3 优先；智谱/OpenAI 可选；Jaccard 与本地 feature-hash 确定性降级
治理闭环	接入→大模型标准识别→向量入库→图谱查重→审核→质量与生命周期→分发→反馈回流
知识与可信	持久化向量表、NetworkX 图谱、SHA-256 前序哈希审计链
企业交付	Windows Waitress、Linux Gunicorn、Docker Compose、健康检查、轮转日志、可选基础认证
动态基线	234→72、69.2%；222→104、53.2%，同数据重复治理结果一致
测试基线	24/24 自动化测试通过：16 项 API 回归 + 8 项故障与降级测试

证据口径 234→72、69.2%、42/22/8 是标准样例可重复结果；222→104、53.2% 是新工业数据动态结果。治理率由输入字段、值分布、模型可用性和阈值配置共同决定，不将任一批次数值写死为通用结果。

能力边界 已接入本地 PaddleOCR、Qwen-VL 与规则解析三级 OCR，但当前仅完成单张工业铭牌验证，尚不能据此宣称全行业图片准确率；SAP/EAM/MES/WMS 仍是可替换模拟适配器；审计能力是单节点 SHA-256 防篡改哈希链，不是公链或多节点联盟链；SQLite 适合单节点试点；互联网部署必须增加 HTTPS、SSO/RBAC 与集中密钥管理。

执行摘要

M-AI Master 面向大型制造集团 ERP、EAM、采购等异构系统中的同物异名、一物多码、属性缺失、标准不一和跨工厂重复建码问题，构建从动态字段接入到工厂反馈回流的 AI 主数据治理 Agent。系统不是单一查重模型，而是将大模型标准识别、持久化向量库、图谱关系、规则约束、人工审核、质量与生命周期、智能分发、反馈再索引组织成可演示、可回放、可验证的八步闭环。

比赛考察重点	系统回答	可验证证据
AI 语义识别能力	在线向量、确定性降级、标准分类、持久化检索和图谱证据共享同一语义底座	/api/semantic、/api/classify、 /api/vectors/search、/api/graph
全流程自动化落地	8 步工作流由后端状态持久化驱动，反馈后重建知识索引并完成闭环	/api/workflow、/api/reviews、 /api/lifecycle、/api/feedback
多工厂多主体价值	集团黄金数据受控下发，工厂成功接收后评价并回流知识库	/api/plants、/api/distribute、 /api/feedback、plant_code
产品落地化	生产 WSGI、可配置路径、日志、认证、Docker 持久卷和跨电脑运行	/api/health、start.bat、 Dockerfile、docker-compose.yml

核心结果

- 标准基线 234 条多源脏数据归并为 72 条黄金主数据，候选压缩率 69.2%。
- 标准基线形成 42 组自动归并、22 组自动新建、8 组人工审核，自动流转覆盖 64/72 个治理组。
- 动态适配基线 222 条工业备件数据归并为 104 条黄金主数据，压缩率 53.2%；连续两次治理结果完全一致。
- 24 项自动化测试全部通过：16 项 API 回归覆盖动态映射、语义降级、真实 OCR 合同、向量库、图谱、审计链、反馈回流、认证与生产部署；8 项故障测试覆盖超时、畸形响应、隔离 OCR Worker、受控安装、无效图片和审计篡改。

比赛要求逐项对照

要求	实现状态	实现说明	演示入口
AI 自动识别主数据标准	已实现	规则属性抽取 + 大模型/降级语义排序，返回标准、置信度、编码前缀	分类推荐
图文智能填报	已实现	隔离 PaddleOCR 真实识别，Qwen-VL 增强与规则兜底；输出字段、置信度、提供方、告警和安装状态	/api/ocr; /api/ocr/install
自然语言驱动分发	已实现	识别目标系统、工厂主体、全量/增量模式和批准范围	分发中心
AI 相似度查重	已实现	在线向量或 Jaccard，叠加硬冲突闸门与图谱关系后聚类	语义检索/治理
物料智能分类推荐	已实现	Top3 候选、置信度、推荐原因、编码前缀和标准名称预览	/api/classify
集团全域同源共治共享	已实现演示	集团/上海/北京三主体、共享黄金数据、范围过滤、审核和分发日志	分发中心/生命周期
八步闭环与知识回流	已实现	分发成功后工厂反馈写入向量元数据、重建索引并完成 workflow	/api/workflow、 /api/feedback
图技术与可信审计	已实现演示	NetworkX 血缘图 + SHA-256 前序哈希审计链，可独立验证	/api/graph、 /api/blockchain/verify

评分呈现建议 上传任意 CSV → 查看动态字段识别与原始扩展字段 → 大模型标准识别 → Embedding 入库与检索 → 图谱查看来源血缘 → 处理多主体审核 → 生命周期批准 → 输入自然语言跨工厂分发 → 提交工厂反馈 → 查看向量刷新、审计链验证和闭环进度。

业务痛点与建设目标

典型问题

- ERP、EAM、采购系统各自建码，同一备件存在中文简称、英文名、品牌型号组合等多种写法。
- 物料名称、描述、品牌、型号、DN、压力、材质混写，字段缺失和单位表达不一致导致规则难以直接比较。
- 仅靠字符串相等无法识别“机械密封/机封”“SKF/斯凯孚”等语义等价表达。
- 集团与工厂之间缺少统一黄金记录、审批责任和分发轨迹，重复采购与重复建码难以量化。
- AI 服务依赖网络和密钥，若没有降级路径，比赛现场或离线环境容易中断整个治理流程。

建设目标

目标层	目标
识别	将名称、描述和扩展属性转化为标准语义表达，返回模型、维度、方法与告警
治理	通过向量召回、规则硬约束、图谱证据和人工审核生成可追溯黄金主数据
知识	将黄金数据与工厂反馈持久化为可重建、可检索的 Embedding 知识索引
可信	通过批次快照、状态机与前序哈希审计链验证关键事件未被篡改
共享	以集团黄金数据为统一来源，按工厂和目标系统执行受控分发并接收反馈
韧性	外部模型不可用时自动降级，服务通过生产 WSGI、健康检查和持久化目录交付
价值	区分固定基线、动态结果、潜在覆盖与实际分发，不把压缩率表述为准确率

总体架构

系统采用同源单页体验、Flask Agent 编排和 SQLite 权威存储。治理结果、向量、图关系所需实体、审计区块、工作流状态和工厂反馈均持久化；IndexedDB 仅作为浏览器缓存。Windows 使用 Waitress，Linux/Docker 使用单进程多线程 Gunicorn，以匹配 SQLite 单节点写入边界。

层级	组件	职责
体验层	单页 HTML	动态接入、总览、知识库、语义检索、图谱、审核、质量、生命周期、分发与反馈
Agent 编排层	Flask REST API + 八步工作流	计划、状态、输入校验、错误返回、角色与工厂范围控制
AI 与规则层	SemanticEngine + AIGovernanceEngine	多模型向量、Jaccard、属性抽取、分类、冲突检测和黄金命名
向量知识层	vector_embeddings	在线/本地向量持久化、统计、重建、余弦检索和反馈增强
图谱与可信层	NetworkX + audit_blocks	来源—黄金—工厂—系统关系、血缘查询、区块哈希验证
治理数据层	SQLite WAL + 外键	批次、记录、黄金、映射、审核、生命周期、质量、分发与反馈
集成层	CSV/OCR/业务系统适配器	多编码接入；模拟 OCR；SAP/EAM/MES/WMS 载荷与日志
运行保障层	Waitress/Gunicorn/Docker	局域网监听、健康检查、轮转日志、可选认证和持久卷

关键数据流

多源接入	AI/向量标准化	图谱/规则治理	审核与生命周期	分发与反馈回流
ERP/EAM/采购/CSV	字段映射、属性抽取、语义向量、分类候选	关系血缘、硬冲突、相似聚类、黄金编码	工厂申请、管理员复核、审批人批准	集团黄金→工厂→业务系统→评价→向量再索引

主数据治理 Agent 八步闭环

步骤	Agent 能力	输入	关键处理	输出/状态
1	多源数据接入	CSV/JSON	编码识别、动态字段映射、扩展字段保留、工厂标准化	批次与原始记录快照
2	大模型标准识别	名称、描述、扩展属性	属性抽取、标准匹配、Top3 分类候选	标准名称、置信度、编码前缀
3	Embedding 向量入库	批次黄金候选	在线模型或 384 维 feature-hash, SQLite BLOB 持久化	可统计、可重建向量索引
4	图谱辅助查重	记录、黄金、映射、主体	NetworkX 构图、血缘关系、硬冲突与相似证据	节点、边、图统计与归并解释
5	多主体人工审核	冲突治理组	范围校验、MERGE/NEW/SKIP、同步映射	审核决策与审计区块
6	质量评估与生命周期	质量指标、申请与变更	角色状态机、正式编码、质量报告	APPROVED/REJECTED/ARCHIVED
7	跨工厂智能分发	中文指令或结构化参数	识别系统、工厂、模式，校验批准范围	分发载荷和成功/失败日志
8	工厂反馈与知识回流	成功分发、评分与意见	前置校验、反馈入库、向量重建、审计追加	closed_loop=true 与知识刷新

闭环特征 每一步都有 API、状态、数据库记录和错误返回。 workflow 进度由当前批次动态计算；审核改变黄金映射，生命周期批准生成正式编码，分发只读取批准数据，反馈必须基于成功分发，并触发向量知识重建和审计链追加，最终完成端到端闭环。

AI 语义识别引擎

多模型与自动降级

SemanticEngine 统一管理模型配置、密钥、请求格式、响应维度校验、缓存和故障降级。首选通义千问 text-embedding-v3，并按当前配置请求 1024 维向量；智谱 embedding-3 与 OpenAI text-embedding-3-small 为可选提供方。在线向量不可用时，语义比较降级为 Jaccard，持久化向量库则使用确定性的 384 维 feature-hash-v1，保证检索与闭环不因网络中断。

场景	处理	返回证据
模型在线	批量调用 Embedding API，计算余弦相似度	provider、model、dimension、method、embedding_active=true
未配置密钥	不发起无效请求，直接进入本地降级	warning=API Key 未配置； dimension=null
超时/非 200/格式错误	捕获异常并返回 None，尝试其他已配置模型，再降级	warning=已配置模型调用失败
降级运行	标准化字符集合的交并比；规则层继续提供型号、口径、压力等约束	method=规则增强 Jaccard； embedding_active=false

相似度公式与决策

- 余弦相似度： $\text{sim}(a,b)=(a \cdot b)/(\|a\| \times \|b\|)$ ，用于在线语义向量比较。
- 降级相似度： $J(A,B)=|A \cap B|/|A \cup B|$ ，字符集合在清洗空格和大小写后计算。
- 治理阈值为 0.85；语义分数不是唯一条件，型号、DN、压力和品类硬冲突优先阻止错误归并，图谱关系用于补充来源与黄金记录之间的可解释证据。
- 所有结果返回实际 method、warning、provider 和 dimension，避免用零填充伪造向量维度。

当前运行事实 未配置 DASHSCOPE_API_KEY 时，语义比较使用规则增强 Jaccard，向量库使用可复现的 384 维 feature-hash-v1。自动化测试模拟通义成功响应，验证 Authorization、text-embedding-v3、1024 维参数和返回契约；系统不使用零向量伪造模型结果。

标准识别、分类推荐与图文填报

标准识别与分类推荐

分类接口不是单纯关键词返回。规则引擎先抽取品牌、型号、DN、压力、材质并识别规则品类；语义引擎再将查询文本与标准品类及同义词文本比较。最终分数由语义分数和规则命中奖励组合，输出 Top3 候选及解释。

输出	含义
recommended_category	SY/T 5497-2018 适配品类
candidates	Top3 候选、分数、编码前缀、推荐原因
attributes	品牌、型号、口径、压力、材质等结构化属性
standard_name_preview	按品类与属性拼装的标准名称预览
semantic	本次实际使用的模型或降级方法
real_ocr / provider / confidence	真实识别标记、实际提供方与识别置信度
engine_status / install_status	隔离运行时、模型、安装阶段、进度与脱敏日志状态

图文智能填报适配器

/api/ocr 支持 multipart 或 base64 图片，按“隔离 PaddleOCR → Qwen-VL → 规则解析”三级路径输出品牌、型号、压力、材质、口径、品类和标准名称预览。Windows 主平台与 Python 3.11 OCR 环境隔离，避免 Python 3.14/oneDNN 兼容问题；/api/ocr/install 提供仅服务器本机可触发的异步安装、进度和脱敏日志。模型或网络不可用时返回明确 warning 并降级，治理流程不中断。

不夸大说明 当前版本已经实现真实 PaddleOCR 图片识别，并用一张工业铭牌完成端到端字段验证；尚未建立足够规模的铭牌图片标注集，因此不宣称通用 OCR 准确率。向量库是 SQLite 持久化精确检索，不是大规模 ANN 集群；图谱是 NetworkX 运行时图，不是 Neo4j 集群；哈希链提供篡改检测，不等同于公链或多节点共识。

AI 相似度查重与黄金主数据生成

算法流程

阶段	处理	风险控制
字段准备	确定性别名评分、一列一字段、自动补全、raw_fields 保留	更换数据集不丢失未识别业务字段
特征准备	清洗文本、同义词归一、属性抽取、标准品类识别	保留原始记录与映射
语义表示	在线向量、Jaccard 或 feature-hash 本地向量	超时、状态码、维度与有限数校验
向量与图谱	持久化索引、来源—黄金关系构图、血缘查询	批次和主体范围过滤
候选聚类	阈值 0.85 的贪婪聚类	品类、型号、DN、压力硬冲突优先
冲突识别	比较材质等关键属性并形成原因	冲突组进入 REVIEW，不自动覆盖
黄金生成	标准名称、稳定前缀与 SHA1 摘要编码	映射、来源、审计和反馈可追溯

三类治理决策

决策	触发条件	后续动作
AUTO_MERGE	同组多条记录且无关键属性冲突	自动形成黄金主数据并保留来源映射
REVIEW	同组存在材质等关键属性冲突或需要人工确认	进入审核池，支持合并、新建或跳过
NEW	记录未与现有候选形成可归并组	形成新黄金数据候选

标准数据形成 42 组自动归并、22 组新建、8 组审核，共 72 个治理组；新工业数据则形成 104 个治理组。结果由当前数据字段、值分布和冲突情况动态计算，不把 234/72 写死到 workflow。

全流程自动化与可审计治理

批次快照

每次接入生成独立 batch_id，原始记录、批次黄金主数据、映射、审核、向量索引、 workflow、图谱查询范围、审计区块和反馈均与批次关联。新批次不会覆盖历史结果，批次快照测试验证历史黄金数据可完整回放。

人工审核

- 审核动作限定为 MERGE、NEW、SKIP，非法动作返回 400。
- 工厂角色只能处理自身可见范围内的审核记录，越权返回 403。
- 合并或新建会同步更新 masters、batch_masters 与 mappings，确保列表、映射和分发读取同一决策。
- 审核记录保留批准动作和时间，未找到或已处理记录返回 404。

生命周期状态机

当前状态	允许转移	角色要求
PENDING	REVIEWED / REJECTED	数据管理员
REVIEWED	APPROVED / REJECTED	审批人
APPROVED	ARCHIVED	数据管理员或审批人
REJECTED / ARCHIVED	无后续转移	终态保护

关键保护

系统拒绝 PENDING→APPROVED 越级审批，并校验审核角色。生命周期申请在批准时生成正式 MDM 编码并写入黄金主数据，使“申请→复核→批准→共享”真正连通。

Embedding 持久化向量库

向量索引以 batch_id、namespace、entity_id、provider 和 model 作为业务维度，保存内容哈希、float32 向量 BLOB、维度和治理元数据。索引支持统计、重建和 TopK 余弦检索；工厂反馈会写入 acceptance_rate、feedback_rating 和 feedback_count 后触发同批次重建。

接口	实现	可验证结果
GET /api/vectors/stats	按命名空间、模型和批次聚合	索引数量、维度、provider/model
POST /api/vectors/rebuild	在线模型或 feature-hash-v1 重建	indexed、warning、元数据
POST /api/vectors/search	查询向量与持久化向量余弦排序	TopK、score、黄金编码与属性

图谱血缘与可验证审计链

NetworkX 将原始记录、黄金主数据、来源系统、工厂和目标系统组织为可查询关系图，用于展示来源血缘和跨主体共享证据。关键治理事件同时写入 audit_blocks：每个区块包含载荷哈希、前序哈希和区块哈希，验证接口可定位高度不连续、前序不匹配、载荷被改写或区块哈希异常。

能力	数据/算法	边界
图谱	NetworkX 节点、边、统计与 lineage 查询	运行时图，不是 Neo4j 集群
审计块	SHA-256(payload)、previous_hash、block_hash	单节点防篡改检测
独立验证	逐块重算载荷、前序和区块头	无多节点共识，不称为公链

多工厂多主体协同治理

工厂选择位于分发中心，批次、记录、映射、审核、生命周期、分发日志和反馈均保留 plant_code/plant_codes。当前主体为集团总部、上海工厂和北京工厂；集团黄金数据可向站点共享，工厂反馈在成功分发后写入向量元数据并刷新知识索引。

主体	数据范围	典型角色	主要动作
集团总部	GROUP_ALL，可查看集团及所有工厂范围	集团数据管理员、集团审批人	维护标准、处理跨工厂冲突、发布黄金数据
上海工厂	PLANT_ONLY，仅上海及可共享集团数据	工厂申请人、工厂数据管理员	提交申请、查看本厂数据、接收集团分发
北京工厂	PLANT_ONLY，仅北京及可共享集团数据	工厂申请人、工厂数据管理员	提交申请、查看本厂数据、接收集团分发

协同价值指标

指标	计算逻辑	解释
共享黄金主数据	plant_codes 覆盖多个主体的黄金记录数	识别跨工厂共同物料
避免重复编码	共享组中 $\Sigma(\max(\text{source_count}-1,0))$	衡量被统一编码吸收的来源重复
潜在工厂交付	批准黄金数据×可接收站点	能力覆盖，不等于已执行任务
实际分发任务	SUCCESS 日志数量及活跃工厂数	真实演示行为计数
反馈接受与评分	accepted、rating、comment 按黄金编码聚合	衡量站点采用情况并进入向量元数据
知识刷新	反馈成功后 vector_embeddings 重建数量	证明分发不是终点而是学习闭环

样例数据归属 234 条标准 CSV 和 222 条工业 CSV 均没有工厂字段，系统按 GROUP 数据处理并允许向站点共享，不把集团数据伪装为上海或北京本地数据。独立多工厂测试使用明确 plant_code 记录验证范围过滤、跨工厂归并、成功分发和反馈前置条件。

工厂反馈与知识回流

反馈不是独立表单：系统要求同一批次、黄金编码和工厂已存在 SUCCESS 分发日志，否则返回 409。有效反馈写入接受状态、评分、意见和操作者，随后刷新向量索引、推进 FEEDBACK 步骤并追加审计区块，使跨工厂共享从单向下发变为“下发—使用—评价—知识更新”。

闭环条件	系统动作	证据
成功分发	记录工厂×系统×黄金编码日志	status=SUCCESS
工厂评价	accepted、rating、comment 入库	plant_feedback
知识更新	反馈文本与评分进入向量元数据并重建	knowledge_refresh.indexed

闭环完成

FEEDBACK=COMPLETED 并追加区块

closed_loop=true、audit_block

自然语言驱动跨系统分发

自然语言分发把用户指令解析为 action、target_systems、plant_code、mode 和 scope，再调用统一分发服务。示例“把上海工厂新增主数据同步到 SAP 和 WMS”会识别 SHANGHAI、SAP/WMS、INCREMENTAL 和 APPROVED_MASTERS；分发成功后可进入工厂反馈与知识回流步骤。

识别维度	规则示例	结果
目标系统	SAP/ERP、EAM/设备、MES/生产、WMS/仓储	规范化为 SAP/EAM/MES/WMS，自动去重
工厂主体	集团、总部、上海工厂、北京工厂等别名	GROUP/SHANGHAI/BEIJING
分发模式	新增、最新、增量、刚批准	INCREMENTAL；其余默认 FULL
业务范围	固定批准主数据	APPROVED_MASTERS

分发安全门

- 目标系统为空或无法识别时拒绝分发。
- 未知 plant_code 返回 400，并返回支持的工厂列表。
- 只允许 AUTO_MERGE、NEW、CONFIRMED_NEW 等批准状态的黄金数据进入载荷。
- 集团黄金数据可分发到站点；站点自有数据只能在相应工厂范围内分发。
- 每个目标系统×黄金编码形成独立日志，记录模式、主体、指令、状态和消息。

集成边界 当前分发适配器生成模拟接口载荷和日志，不宣称已调用真实 SAP RFC/OData 或 EAM/MES/WMS；工厂反馈基于模拟成功分发。生产接入时替换系统适配器，不改变审批、范围、幂等、日志、反馈和审计合同。

数据可信、向量知识与系统韧性

权威数据与缓存

- SQLite 是后端权威数据源；前端 IndexedDB 仅用于展示缓存。
- 启用 WAL、外键和索引，批次与明细通过 batch_id 关联。
- 删除全部数据必须提交 confirm=DELETE_ALL_DATA，避免误操作。
- CSV 接入支持 UTF-8、UTF-8 BOM、GBK；字段映射采用标准化别名、确定性评分和一系列字段约束。缺失的系统来源与创建时间自动补全，未映射的子类、价格、库存、供应商、规格型号和状态保存在 raw_fields，避免更换数据集时丢字段。

动态字段映射与数据集适配

版本不假定每份 CSV 都包含固定的系统来源、品类和创建时间。前后端使用一致的标准化别名与确定性评分；一个 CSV 列最多映射到一个标准字段。缺失系统来源时填充“CSV 导入”，缺失创建时间时使用导入时间，所有未使用列原样保存在 raw_fields 并参与子类、型号、规格、品牌和材质等语义增强。

数据集	识别与治理结果	稳定性证据
标准样例 234 条	72 黄金；69.2%；42 归并/22 新建/8 审核	端到端固定基线
工业样例 222 条	104 黄金；53.2%；32 归并/67 新建/5 审核	连续两次 summary 完全一致

质量评估

系统计算完整性、准确性代理指标、一致性、唯一性和标准化程度，并按来源系统汇总问题及建议。accuracy 是基于结构异常数量的质量代理分数，不等同于人工真值集上的模型准确率。独立的 50 条、27 组人工真值集通过 Pairwise Precision/Recall/F1 与 B³ 指标评估聚类质量；结果只代表当前样本，不外推为全域准确率。

故障韧性

故障	系统行为	结果
API Key 缺失	语义使用 Jaccard，向量库使用 feature-hash	治理与检索不中断
模型超时/非 200	捕获异常，尝试其他模型后降级	返回 warning，不崩溃
模型维度/数值错误	拒绝不合格向量，不进行零填充	降级并留痕
字段缺失或更换数据集	自动补全来源/时间，扩展字段进入 raw_fields	动态治理结果可重复
非法状态转移/越权工厂	返回 409/403	阻止脏状态写入
无成功分发即反馈	返回 409	防止虚假知识回流
审计链被修改	重新计算载荷、前序和区块哈希	valid=false 并返回错误高度
主程序与 Paddle/oneDNN 不兼容	调用隔离 Python 3.11 OCR Worker，主服务不加载冲突运行时	真实识别继续，主平台无需重启

故障	系统行为	结果
局域网远程触发 OCR 安装	校验 remote_addr 与转发头，只允许服务器回环地址	返回 403，阻止远程执行安装命令
OCR 安装中断或残留锁	异步进度、锁超时回收、日志摘要与多源下载	失败可诊断并继续使用其他识别路径

量化结果与业务价值

标准测试基线

指标	结果	计算/含义
标准基线原始记录	234	备品备件脏数据.csv
标准基线黄金主数据	72	干净测试库可重复结果
标准候选压缩率	69.2%	$(1-72/234) \times 100\%$ ，不是准确率
标准自动归并	42	多源同物且无关键冲突
标准自动新建	22	未形成可归并候选
标准人工审核	8	关键属性冲突或需要确认
标准自动流转覆盖	88.9%	$(42+22)/72$ ，仅描述治理组
动态基线原始记录	222	industrial_parts_dirty_data.csv
动态基线黄金主数据	104	按新字段和值分布动态计算
动态候选压缩率	53.2%	$(1-104/222) \times 100\%$
稳定性	两次结果相同	104 黄金、5 审核、32 归并、67 新建
人工真值集	50 条 / 27 组	数据集 SHA-256 固定，覆盖率 100%
当前阈值 0.55	Pairwise F1 61.29%	P=48.72%，R=82.61%；B ³ F1=85.58%
校准建议阈值 0.60	Pairwise F1 73.91%	P=R=73.91%；B ³ F1=90.59%，仅作参考

三类业务价值

价值方向	可量化表现	管理意义
同源	不同批次来源记录统一映射到批次黄金记录	减少系统代码歧义并保留历史快照
共治	标准样例仅 8/72 组进入人工审核	专家集中处理真正冲突
共享	批准黄金数据按工厂与目标系统受控分发	集团标准复用到站点且过程可追踪
回流	工厂评分和意见进入向量元数据并触发重建	站点使用经验反哺集团知识
韧性 & 交付	无 Key 可运行；Waitress/Gunicorn/Docker 可跨电 脑部署	降低弱网中断和环境迁移风险

指标纪律 已建立 50 条、27 组人工真值集，报告 Pairwise 与 B³ 指标、阈值扫描、数据集哈希和错误样例。压缩率、质量代理分数和真值集 F1 分开呈现；当前阈值建议只适用于该样本，新数据域必须重新抽样标注和校准。

二十项比赛亮点

序号	亮点	技术或业务证据
1	在线语义 + 离线确定性双路径	余弦、Jaccard 与 feature-hash 自动切换，流程不中断
2	多模型提供方统一合同	通义优先，智谱/OpenAI 可选，返回真实 provider/model/dimension
3	动态字段识别与扩展字段保留	确定性评分、一列一字段、自动补全、raw_fields
4	持久化 Embedding 向量库	SQLite BLOB、内容哈希、元数据、统计、重建与检索
5	NetworkX 图谱辅助治理	来源—黄金—工厂—系统关系与单编码血缘查询
6	硬约束守护语义模型	品类、型号、DN、压力冲突优先阻断错误归并
7	八步 Agent 闭环	接入→识别→向量→图谱→审核→生命周期→分发→反馈
8	Agent 计划与可追踪状态	/api/agent/plan、workflow_steps、动态进度
9	不可变批次快照	历史批次独立回放，不被新批次覆盖
10	可验证哈希审计链	载荷哈希、前序哈希、区块哈希和独立验证接口
11	角色化生命周期状态机	管理员复核、审批人批准，拒绝越级流转
12	集团共享与工厂范围并存	GROUP 黄金可下发，站点数据保持本地范围
13	自然语言三要素解析	同时识别系统、工厂和全量/增量模式
14	工厂反馈驱动知识回流	成功分发前置、评价聚合、向量刷新、闭环完成
15	质量与业务指标纪律	固定/动态基线、潜在/实际行为分开计算
16	生产级 Windows/Linux WSGI	Waitress 与单进程多线程 Gunicorn
17	Docker 与跨电脑交付	Compose、持久卷、相对 /api、局域网地址
18	运行可观测与安全基线	健康检查、Trace ID、响应耗时、日志轮转、可选认证
19	真实 OCR 三级适配与隔离运行时	PaddleOCR 真实识别、Qwen-VL 增强、规则兜底；Python 3.11 Worker 与受控安装
20	24 项自动化测试	16 项 API 回归 + 8 项故障测试，覆盖 AI、OCR、动态数据、向量、图谱、审计、反馈、认证与部署

REST API 与模块映射

模块	方法与路径	主要输出
健康与能力	GET /api/health; GET /api/agent/capabilities	4.3、就绪、存储、模型、OCR、工厂、部署和认证状态
Agent 与 workflow	POST /api/agent/plan; GET /api/workflow/{batch_id}	计划、八步状态、进度、closed_loop
批次接入	POST /api/upload; POST /api/batches	动态映射、批次、记录、黄金、映射、审核与质量
语义与分类	POST /api/semantic; POST /api/classify	相似度、模型元数据、Top3 分类候选
治理演示	POST /api/govern; POST /api/governance	动态治理摘要与当前批次状态
图文填报	POST /api/ocr; GET/POST /api/ocr/install	真实/降级标记、提供方、字段、置信度、安装阶段、进度与日志摘要
向量知识	GET /api/vectors/stats; POST /api/vectors/rebuild; POST /api/vectors/search	索引、维度、模型、TopK 余弦结果
图谱血缘	GET /api/graph; GET /api/graph/lineage/{mdm_code}	节点、边、统计和单编码血缘
可信审计	GET /api/blockchain/blocks; GET /api/blockchain/verify	区块列表、链有效性、错误位置
搜索	POST /api/search	条件、语义元数据、结果和无精确匹配建议
审核与生命周期	GET/PATCH /api/reviews; POST/PATCH /api/lifecycle	决策、角色校验、正式编码和状态轨迹
工厂与分发	GET /api/plants; POST /api/intent; POST /api/distribute	主体指标、意图解析、分发日志
反馈回流	POST /api/feedback	评价、向量刷新、审计区块、闭环状态
保护接口	DELETE /api/data; DELETE /api/search-history; DELETE /api/distribution-logs	确认令牌或指定日志清理
人工真值评测	GET /api/evaluation/ground-truth; POST /api/evaluation/run	Pairwise/B ³ 指标、阈值扫描、数据集哈希、错误样例与校准建议
可解释治理	POST /api/explain	真实来源记录、关键属性、冲突、命中规则与大模型/模板解释

部署、配置与验证

运行与跨电脑交付步骤

步骤	命令/动作	说明
1	复制 backend/.env.example 为 backend/.env (可选)	配置 API Key、端口、数据/日志路径和认证
2	Windows 双击 start.bat; Linux 执行 ./start.sh	自动创建主运行环境并启动 Waitress/Gunicorn; 平台可先 在无 OCR 依赖状态运行
3	服务器本机首次点击“识别并标准化”，或 手动运行 install-ocr.bat	Windows 自动安装隔离 Python 3.11、PaddleOCR CPU 依赖与模型；完成后自动继续识别
4	本机访问 127.0.0.1:5000; 其他电脑访问启动窗口 LAN 地址	前端使用相对 /api; 远程客户端不能触发 OCR 安装，但可 使用服务器已经安装的 OCR
5	访问 /api/health; 运行 test_api.py 与 test_chaos.py	确认 ready=true、version=4.3、16/16 API 回归和 8/8 故障测试
6	docker compose up -d --build (可选)	数据库和日志持久化到 runtime/data、runtime/logs; OCR 镜像按需构建

环境变量

变量	用途	缺失时行为
DASHSCOPE_API_KEY	通义 text-embedding-v3	语义降级，流程继续
ZHIPU_API_KEY	智谱 embedding-3 备选	跳过该提供方
OPENAI_API_KEY	OpenAI embedding 备选	跳过该提供方
MDM_HOST / MDM_PORT	监听地址与端口	0.0.0.0 / 5000
MDM_THREADS	Waitress/Gunicorn 线程数	8
MDM_PRODUCTION	启用生产 WSGI	start 脚本设为 1
MDM_DB_PATH	SQLite 持久化路径	兼容 backend/mdm_data.db 或 runtime/data
MDM_LOG_DIR	轮转日志目录	不配置则仅控制台日志
MDM_AUTH_USER / PASSWORD	可选基础认证	密码为空时关闭认证
MDM_MAX_RECORDS	单批记录上限	10000
MDM_MAX_UPLOAD_BYTES	上传体积上限	20 MiB
MDM_PADDLEOCR_ENABLED	启用本地 PaddleOCR 路径	默认启用；失败时转 Qwen-VL/规则
MDM_ALLOW_OCR_INSTALL	允许服务器本机触发 OCR 自动安装	默认 1；设为 0 时只能手动安装

变量	用途	缺失时行为
MDM_PIP_INDEX_URL	企业内部或指定 Python 包镜像	安装脚本尝试阿里云、清华和官方源
MDM_OCR_WORKER_TIMEOUT	隔离 OCR Worker 超时秒数	默认 180 秒，超时后进入降级路径
MDM_MAX_OCR_BYTES	单张 OCR 图片大小上限	默认 10 MiB，超限返回 413

本次验证结果

- 动态字段映射、非标准字段保留与重复治理稳定性测试：通过。
- 标准 234 条端到端流程、批次快照和删除保护测试：通过。
- 通义请求契约、语义降级、分类与 OCR 接口测试：通过。
- 多工厂范围、集团黄金数据分发和自然语言意图测试：通过。
- 持久化向量入库、统计、重建与检索测试：通过。
- NetworkX 图谱、黄金主数据血缘查询测试：通过。
- 哈希审计链、成功分发前置条件、工厂反馈与知识回流测试：通过。
- 运行结果：test_api.py 为 16/16，test_chaos.py 为 8/8，总计 24/24，全部 OK。真实工业铭牌端到端验证识别置信度 0.9951，并结构化得到 SKF、6312-2RS1、316L、1.6MPa 与 DN80。

企业交付现状与演进路线

当前 能力	仍需补齐的生产边界	建议演进
Waitress/Gunicorn + Docker + SQLite WAL	SQLite 仍是单节点写入和有限并发	迁移 PostgreSQL，建立迁移、备份、恢复和高可用
基础认证、Trace ID、轮转日志	尚未接入 SSO/RBAC、HTTPS 与集中日志	接入企业 IAM、网关 TLS、SIEM 与密钥管理
SQLite 持久化精确向量检索	大规模 TopK 需要 ANN 和分片	接入 pgvector/Milvus/企业向量平台
NetworkX 运行时图谱	缺少独立图数据库和跨域图查询	接入 Neo4j/JanusGraph 并定义图数据治理
SHA-256 前序哈希审计链	单节点存储，无多方共识和硬件签名	接入联盟链/KMS 签名或 WORM 审计存储
隔离 Python 3.11 PaddleOCR + Qwen-VL + 规则兜底，并提供本机受控一键安装	当前仅验证一张英文工业铭牌；模型安装依赖网络/制品源，尚无大规模图片真值集和 GPU 服务	建设多版式铭牌标注集，报告字段级 Precision/Recall；企业内网使用制品库并按需接入 Qwen-VL/GPU 推理
分发模拟适配器	没有真实 SAP/EAM/MES/WMS 联调	建立 OData/RFC/REST 适配器、幂等、重试和回执
50 条、27 组人工归并真值集 + 两套可重复治理基线	样本规模仍小，Pairwise F1 尚有提升空间，未形成跨数据域与长期业务工时证据	扩展分层金标准集，按品类/工厂报告 Precision、Recall、F1、置信区间与真实节省工时

演进原则 保持现有 REST 合同、八步状态机、批次范围、向量元数据、图谱标识、审计事件和反馈合同稳定，逐步替换模型、数据库、图平台、区块链和业务系统适配器；避免为了包装新技术破坏已验证的业务闭环。

核心实现代码

以下代码来自 GitHub 提交 674c4d2 的 backend/app.py，用于展示比赛重点核心实现。完整可运行源码以 backend/app.py、backend/index.html、backend/ocr_worker.py、requirements.txt、requirements-ocr.txt、install-ocr.bat、start.bat 和 docker-compose.yml 为准。

语义 API：在线向量与降级返回

backend/app.py - /api/semantic

```
@app.post("/api/semantic")
def api_semantic():
    payload = request.get_json(silent=True) or {}
    text1 = _clean_value(payload.get("text1"))
    text2 = _clean_value(payload.get("text2"))
    requested_model = _clean_value(payload.get("model")) or "qwen"
    if not text1 or not text2:
        return jsonify({"error": "text1 and text2 are required"}), 400
    if requested_model not in SemanticEngine.MODELS:
        return jsonify({
            "error": f"unsupported model: {requested_model}",
            "supported_models": list(SemanticEngine.MODELS),
        }), 400

    vectors, metadata = semantic_engine.resolve_embeddings(
        [text1, text2], requested_model
    )
    if vectors is None:
        similarity = semantic_engine.jaccard_similarity(text1, text2)
        return jsonify({
            "similarity": round(similarity, 6),
            **metadata,
            "requested_model": requested_model,
            "text1": text1,
            "text2": text2,
        })
    return jsonify({
        "similarity": round(
            semantic_engine.cosine_similarity(vectors[0], vectors[1]), 6
        ),
        **metadata,
        "requested_model": requested_model,
        "text1": text1,
        "text2": text2,
    })
```

模型请求容错与维度校验

backend/app.py - SemanticEngine._request_embeddings

```
def _request_embeddings(self, texts, model_key="qwen"):
    config = self.MODELS.get(model_key)
    api_key = self.api_keys.get(model_key, "")
    if not config or not api_key or not texts:
        return None
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
    }
    values = [text[:2048] for text in texts]
    payload = {
        "model": config["model"],
        "input": {"texts": values},
        "parameters": {
            "dimension": config["dimension"],
            "text_type": "document",
        },
    }
    try:
        response = requests.post(
            config["url"], headers=headers, json=payload, timeout=self.timeout
        )
        if response.status_code != 200:
            return None
        data = response.json()
        items = sorted(
            data["output"]["embeddings"],
            key=lambda item: int(item.get("text_index", 0)),
        )
        vectors = [
            [float(value) for value in item["embedding"]] for item in items
        ]
        if len(vectors) != len(values):
            return None
        if any(len(vector) != config["dimension"] for vector in vectors):
            return None
        if any(
            not np.isfinite(np.asarray(vector, dtype=float)).all()
            for vector in vectors
        ):
            return None
        return vectors
    except (
        requests.Timeout,
        requests.RequestException,
        KeyError,
        TypeError,
        ValueError,
        IndexError,
    ):
        return None
```

自然语言分发意图

backend/app.py - _parse_distribution_intent

```
def _parse_distribution_intent(text: str, plant_code=None) -> dict:
    lower = _clean_value(text).lower()
    rules = {
        "SAP": ("sap", "erp", "财务系统"),
        "EAM": ("eam", "设备系统", "资产系统"),
        "MES": ("mes", "制造系统", "生产系统"),
        "WMS": ("wms", "仓储系统", "库存系统", "仓库系统"),
    }
    targets = [
        target
        for target, keywords in rules.items()
        if any(keyword in lower for keyword in keywords)
    ]
    detected_plant = _normalize_plant_code(plant_code)
    for alias, code_value in PLANT_ALIASES.items():
        if alias in lower:
            detected_plant = code_value
            break
    mode = "INCREMENTAL" if any(
        key in lower for key in ("增量", "新增", "刚批准", "最新")
    ) else "FULL"
    return {
        "text": text,
        "action": "DISTRIBUTE" if targets else "UNKNOWN",
        "target_systems": targets,
        "plant_code": detected_plant,
        "mode": mode,
        "scope": "APPROVED_MASTERS",
        "confidence": 0.96 if targets else 0.35,
    }
```

生命周期状态机

backend/app.py - update_lifecycle 核心状态定义

```
transitions = {
    "PENDING": {"REVIEWED", "REJECTED"},
    "REVIEWED": {"APPROVED", "REJECTED"},
    "APPROVED": {"ARCHIVED"},
    "REJECTED": set(),
    "ARCHIVED": set(),
}
required_reviewers = {
    ("PENDING", "REVIEWED"): {"数据管理员"},
    ("PENDING", "REJECTED"): {"数据管理员"},
    ("REVIEWED", "APPROVED"): {"审批人"},
    ("REVIEWED", "REJECTED"): {"审批人"},
    ("APPROVED", "ARCHIVED"): {"数据管理员", "审批人"},
}
if status != current_status:
    if status not in transitions.get(current_status, set()):
        return jsonify({
            "error": f"invalid lifecycle transition: {current_status} -> {status}"
        }), 409
    allowed_reviewers = required_reviewers.get(
        (current_status, status), set()
    )
    if allowed_reviewers and reviewer not in allowed_reviewers:
        return jsonify({"error": "reviewer role is not allowed"}), 403
```

八步工作流定义

backend/app.py - WORKFLOW_DEFINITION

```
WORKFLOW_DEFINITION = [
    ("INGEST", 1, "多源数据接入", "/api/upload"),
    ("STANDARDIZE", 2, "大模型标准识别", "/api/classify"),
    ("VECTOR_INDEX", 3, "Embedding 向量入库", "/api/vectors/rebuild"),
    ("GRAPH_DEDUPE", 4, "图谱辅助查重", "/api/graph"),
    ("REVIEW", 5, "多主体人工审核", "/api/reviews"),
    ("QUALITY", 6, "质量评估与生命周期", "/api/lifecycle"),
    ("DISTRIBUTE", 7, "跨工厂智能分发", "/api/distribute"),
    ("FEEDBACK", 8, "工厂反馈与知识回流", "/api/feedback"),
]
```

持久化向量入库

backend/app.py - _index_batch_vectors

```
def _index_batch_vectors(
    conn: sqlite3.Connection, batch_id: str, preferred_model: str = "local", namespace: str = "golden_master"
) -> dict:
    rows = conn.execute("SELECT * FROM batch_masters WHERE batch_id = ? ORDER BY rowid", (batch_id,)).fetchall()
    if not rows:
        return {"batch_id": batch_id, "indexed": 0, "namespace": namespace, "provider": "local",
            "model": "feature-hash-v1", "dimension": 384, "warning": "当前批次没有黄金主数据。"}
    feedback_rows = {
        row["mdm_code"]: dict(row) for row in conn.execute(
            """SELECT mdm_code, AVG(rating) AS feedback_rating, AVG(accepted) AS acceptance_rate,
                GROUP_CONCAT(comment, ' ') AS feedback_text, COUNT(*) AS feedback_count
                FROM plant_feedback WHERE batch_id = ? GROUP BY mdm_code""", (batch_id,)
        ).fetchall()
    }
    contents = []
    for row in rows:
        feedback_item = feedback_rows.get(row["mdm_code"], {})
        feedback_text = _clean_value(feedback_item.get("feedback_text"))
        content = _master_vector_content(row)
        contents.append(f"{content} 工厂反馈 {feedback_text}".strip() if feedback_text else content)
    vectors, metadata = semantic_engine.resolve_vectorization(contents, preferred_model, "document")
    now = _utc_now()
    for row, content, vector in zip(rows, contents, vectors):
        vector_array = np.asarray(vector, dtype=np.float32)
        item_metadata = {
            "standard_name": row["standard_name"], "category": row["category"],
            "brand": row["brand"], "model": row["model"], "dn": row["dn"],
            "pressure": row["pressure"], "material": row["material"],
            "decision": row["decision"], "plant_codes": row["plant_codes"],
            "feedback_count": int(feedback_rows.get(row["mdm_code"], {}).get("feedback_count") or 0),
            "feedback_rating": round(float(feedback_rows.get(row["mdm_code"], {}).get("feedback_rating") or 0), 2),
            "acceptance_rate": round(float(feedback_rows.get(row["mdm_code"], {}).get("acceptance_rate") or 0), 4),
        }
        conn.execute(
            """INSERT INTO vector_embeddings
            (namespace, entity_type, entity_id, batch_id, plant_code, content, content_hash,
            vector, dimension, provider, model, metadata, created_at, updated_at)
            VALUES (?, 'MASTER', ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(namespace, batch_id, entity_type, entity_id, provider, model) DO UPDATE SET
            plant_code=excluded.plant_code, content=excluded.content, content_hash=excluded.content_hash,
            vector=excluded.vector, dimension=excluded.dimension, metadata=excluded.metadata,
            updated_at=excluded.updated_at""",
            (namespace, row["mdm_code"], batch_id, row["plant_codes"] or "GROUP", content,
            hashlib.sha256(content.encode("utf-8")).hexdigest(), vector_array.tobytes(), len(vector_array),
            metadata["provider"], metadata["model"], json.dumps(item_metadata, ensure_ascii=False), now, now),
        )
    return {"batch_id": batch_id, "namespace": namespace, "indexed": len(rows), **metadata}
```

哈希审计块与独立验证

backend/app.py - _append_audit_block / _verify_audit_chain

```
def _append_audit_block(
    conn: sqlite3.Connection, batch_id: str, event_type: str, entity_type: str,
    entity_id: str, payload: dict, actor: str = "M-AI Agent",
) -> dict:
    previous = conn.execute(
        "SELECT height, block_hash FROM audit_blocks WHERE batch_id = ? ORDER BY height DESC LIMIT 1", (batch_id,)
    ).fetchone()
    height = int(previous["height"]) + 1 if previous else 1
    previous_hash = previous["block_hash"] if previous else "0" * 64
    payload_json = _canonical_json(payload)
    payload_hash = hashlib.sha256(payload_json.encode("utf-8")).hexdigest()
    merkle_root = payload_hash
    created_at = _utc_now()
    header = {
        "batch_id": batch_id, "height": height, "event_type": event_type,
        "entity_type": entity_type, "entity_id": entity_id, "actor": actor,
        "payload_hash": payload_hash, "previous_hash": previous_hash,
        "merkle_root": merkle_root, "created_at": created_at,
    }
    block_hash = hashlib.sha256(_canonical_json(header).encode("utf-8")).hexdigest()
    conn.execute(
        """INSERT INTO audit_blocks
        (batch_id, height, event_type, entity_type, entity_id, actor, payload_json,
        payload_hash, previous_hash, merkle_root, block_hash, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (batch_id, height, event_type, entity_type, entity_id, actor, payload_json,
        payload_hash, previous_hash, merkle_root, block_hash, created_at),
    )
    return {**header, "block_hash": block_hash}

def _verify_audit_chain(conn: sqlite3.Connection, batch_id: str) -> dict:
    blocks = _rows(conn, "SELECT * FROM audit_blocks WHERE batch_id = ? ORDER BY height", (batch_id,))
    expected_previous = "0" * 64
    errors = []
    for expected_height, block in enumerate(blocks, 1):
        payload_hash = hashlib.sha256((block["payload_json"] or "").encode("utf-8")).hexdigest()
        header = {
            "batch_id": block["batch_id"], "height": block["height"], "event_type": block["event_type"],
            "entity_type": block["entity_type"], "entity_id": block["entity_id"], "actor": block["actor"],
            "payload_hash": block["payload_hash"], "previous_hash": block["previous_hash"],
            "merkle_root": block["merkle_root"], "created_at": block["created_at"],
        }
        calculated_hash = hashlib.sha256(_canonical_json(header).encode("utf-8")).hexdigest()
        if int(block["height"]) != expected_height:
            errors.append({"height": block["height"], "error": "区块高度不连续"})
        if block["previous_hash"] != expected_previous:
            errors.append({"height": block["height"], "error": "前序哈希不匹配"})
        if payload_hash != block["payload_hash"] or block["merkle_root"] != block["payload_hash"]:
            errors.append({"height": block["height"], "error": "业务载荷哈希不匹配"})
        if calculated_hash != block["block_hash"]:
            errors.append({"height": block["height"], "error": "区块哈希不匹配"})
        expected_previous = block["block_hash"]
    return {
        "batch_id": batch_id, "valid": not errors, "block_count": len(blocks),
        "latest_hash": expected_previous if blocks else None, "errors": errors,
        "chain_type": "本地联盟链式审计账本", "public_chain": False,
    }
```

测试证据与代码定位

能力	代码位置	测试位置
动态字段映射与扩展保留	backend/app.py:75、123、193 起	test_api.py:75 起
语义模型、超时与降级	backend/app.py:1046、2156 起	test_api.py:247、301 起
八步 Agent 工作流	backend/app.py:1415、2242、2281 起	test_api.py:114、461 起
持久化向量库	backend/app.py:1536、2299 起	test_api.py:413 起
NetworkX 图谱与血缘	backend/app.py:2349 起	test_api.py:435 起
审计链与反馈回流	backend/app.py:1470、2387、2902 起	test_api.py:461 起
多工厂协同与分发	backend/app.py:2095、2461、2824 起	test_api.py:324、372 起
生命周期状态机	backend/app.py:2707、2734 起	test_api.py:114 起
生产服务与健康检查	backend/app.py:64、2013、3006 起	test_api.py:50、61 起
部署与持久化合同	start.bat、start.sh、Dockerfile、docker-compose.yml	test_api.py:61 起
真实 OCR、隔离 Worker 与受控安装	backend/app.py:1255-1572、2998-3083; backend/ocr_worker.py	test_api.py、test_chaos.py 的 OCR 合同、Worker、安装权限与降级测试
人工真值集与阈值校准	backend/app.py:3086-3240; evaluation/ground_truth_50.csv	test_api.py 的 ground-truth/evaluation 测试

结论 M-AI Master 4.3 已形成可运行的八步治理闭环：可调用且可降级的 AI 语义能力、真实 OCR 三级识别、可持久化向量知识、可解释图谱血缘、可验证哈希审计、人工真值评测、多工厂分发和反馈回流，并具备单节点企业试点交付能力。真实 SAP/EAM/MES/WMS 连接器、企业 SSO、高可用数据库、独立图平台、多方共识链和跨数据域大样本评测仍属于后续生产化工作。